

LAPORAN PRAKTIKUM STRUKTUR DATA & ALGORITMA

MODUL 6: STRUKTUR DATA LINKED LIST

Nama	Rafi hamzah
Nim	2025573010115
Kelas	1D
Dosen	M.REZA ZULMAN, S.ST. M.Sc

1. Setup Project & Inisialisasi Folder

Langkah awal praktikum dilakukan dengan membuat struktur folder praktikum-6/ di dalam repositori GitHub lokal dan melakukan inisialisasi npm project menggunakan terminal.

```
# Masuk ke root repository
cd nama-repository-kamu

# Membuat folder praktikum-6 dan masuk ke dalamnya
mkdir praktikum-6
cd praktikum-6

# Inisialisasi npm project
npm init -y
```

2. Bagian 1: Implementasi Singly Linked List Dasar & Latihan 1

Pada bagian ini, dibuat berkas bernama 01-linked-list.js. Di dalamnya diimplementasikan class Node dan class LinkedList dasar beserta seluruh fitur tambahan yang diminta pada Latihan 1 (getAt, deleteAt, indexOf, isEmpty, dan clear).

Kode Program (01-linked-list.js)

```
// 01-linked-list.js
// SINGLY LINKED LIST - IMPLEMENTASI LENGKAP & LATIHAN 1

class Node {
  constructor(data) {
    this.data = data; // Nilai yang disimpan
    this.next = null; // Pointer ke node berikutnya
```

```

    }
}

class LinkedList {
    constructor() {
        this.head = null; // Pointer ke node pertama
        this.size = 0;    // Jumlah node
    }

    // Tambah node di AKHIR - O(n)
    append(data) {
        const newNode = new Node(data);
        if (!this.head) {
            this.head = newNode;
        } else {
            let current = this.head;
            while (current.next) {
                current = current.next;
            }
            current.next = newNode;
        }
        this.size++;
    }

    // Tambah node di AWAL - O(1)
    prepend(data) {
        const newNode = new Node(data);
        newNode.next = this.head;
        this.head = newNode;
        this.size++;
    }

    // Insert di posisi indeks tertentu - O(n)
    insertAt(data, index) {
        if (index < 0 || index > this.size) {
            console.log('Index di luar batas!');
            return;
        }
        if (index === 0) {
            this.prepend(data);
            return;
        }
    }
}

```

```

    const newNode = new Node(data);
    let current = this.head;
    for (let i = 0; i < index - 1; i++) {
        current = current.next;
    }
    newNode.next = current.next;
    current.next = newNode;
    this.size++;
}

// Hapus node berdasarkan nilai - O(n)
delete(data) {
    if (!this.head) return false;
    if (this.head.data === data) {
        this.head = this.head.next;
        this.size--;
        return true;
    }
    let current = this.head;
    while (current.next) {
        if (current.next.data === data) {
            current.next = current.next.next;
            this.size--;
            return true;
        }
        current = current.next;
    }
    return false;
}

// Cari node berdasarkan nilai - O(n)
search(data) {
    let current = this.head;
    let index = 0;
    while (current) {
        if (current.data === data) return index;
        current = current.next;
        index++;
    }
    return -1;
}

```

```

// Balik urutan list - O(n)
reverse() {
    let prev = null;
    let current = this.head;
    while (current) {
        const next = current.next;
        current.next = prev;
        prev = current;
        current = next;
    }
    this.head = prev;
}

// Tampilkan semua node - O(n)
print() {
    if (!this.head) {
        console.log('[List kosong]');
        return;
    }
    let result = '';
    let current = this.head;
    while (current) {
        result += `[\\${current.data}]` + (current.next ? ' -> ' :
    '');

        current = current.next;
    }
    console.log(`\\${result} (size: \\${this.size})`);
}

// =====
// LATIHAN 1: FITUR TAMBAHAN LINKED LIST
// =====

getAt(index) {
    if (index < 0 || index >= this.size) return null;
    let current = this.head;
    for (let i = 0; i < index; i++) {
        current = current.next;
    }
    return current.data;
}

```

```

deleteAt(index) {
    if (index < 0 || index >= this.size) return null;
    let current = this.head;
    if (index === 0) {
        const deletedData = this.head.data;
        this.head = this.head.next;
        this.size--;
        return deletedData;
    }
    for (let i = 0; i < index - 1; i++) {
        current = current.next;
    }
    const deletedData = current.next.data;
    current.next = current.next.next;
    this.size--;
    return deletedData;
}

indexOf(data) {
    return this.search(data);
}

isEmpty() {
    return this.size === 0;
}

clear() {
    this.head = null;
    this.size = 0;
    console.log('List berhasil dikosongkan.');
```

}

// Pengujian
const ll = new LinkedList();
console.log('=== Append ===');
ll.append(10); ll.append(20); ll.append(30); ll.append(40);
ll.print();

console.log('\n=== Prepend ===');
ll.prepend(5);
ll.print();

```

console.log('\n=== Insert di indeks 2 ===');
ll.insertAt(15, 2);
ll.print();

console.log('\n=== Search & IndexOf ===');
console.log('Indeks nilai 20:', ll.search(20));
console.log('Indeks nilai 30 (indexOf):', ll.indexOf(30));

console.log('\n=== Delete 20 ===');
ll.delete(20);
ll.print();

console.log('\n=== Reverse ===');
ll.reverse();
ll.print();

console.log('\n=== Uji Latihan 1 ===');
console.log('Data di indeks 2:', ll.getAt(2));
console.log('Hapus data di indeks 1:', ll.deleteAt(1));
ll.print();

console.log('\n=== Uji Clear ===');
ll.clear();
ll.print();

```

Output Terminal (01-linked-list.js)

```

=== Append ===
[10] -> [20] -> [30] -> [40] (size: 4)

=== Prepend ===
[5] -> [10] -> [20] -> [30] -> [40] (size: 5)

=== Insert di indeks 2 ===
[5] -> [10] -> [15] -> [20] -> [30] -> [40] (size: 6)

=== Search & IndexOf ===
Indeks nilai 20: 3
Indeks nilai 30 (indexOf): 4

=== Delete 20 ===

```

```

[5] -> [10] -> [15] -> [30] -> [40] (size: 5)

=== Reverse ===
[40] -> [30] -> [15] -> [10] -> [5] (size: 5)

=== Uji Latihan 1 ===
Data di indeks 2: 15
Hapus data di indeks 1: 30
[40] -> [15] -> [10] -> [5] (size: 4)

=== Uji Clear ===
List berhasil dikosongkan.
[List kosong]

```

3. Bagian 2: Operasi Lanjutan (Deteksi Siklus & Merge Terurut)

Bagian ini mengimplementasikan algoritma penunjuk Floyd (Fast & Slow Pointer) untuk mendeteksi siklus/looping sirkular pada memori list serta fungsi untuk menggabungkan dua buah Linked List terurut menjadi satu.

Kode Program (02-11-lanjutan.js)

```

// 02-11-lanjutan.js
class Node {
  constructor(d) {
    this.data = d;
    this.next = null;
  }
}

function adaSiklus(head) {
  let lambat = head;
  let cepat = head;
  while (cepat && cepat.next) {
    lambat = lambat.next;
    cepat = cepat.next.next;
    if (lambat === cepat) return true;
  }
  return false;
}

function mergeTerurut(head1, head2) {
  const dummy = new Node(0);
  let current = dummy;

```

```

    while (head1 && head2) {
        if (head1.data <= head2.data) {
            current.next = head1;
            head1 = head1.next;
        } else {
            current.next = head2;
            head2 = head2.next;
        }
        current = current.next;
    }
    current.next = head1 || head2;
    return dummy.next;
}

function buatList(arr) {
    if (!arr.length) return null;
    const head = new Node(arr[0]);
    let cur = head;
    for (let i = 1; i < arr.length; i++) {
        cur.next = new Node(arr[i]);
        cur = cur.next;
    }
    return head;
}

function cetakList(head) {
    let s = '';
    let cur = head;
    while (cur) {
        s += `[\\${cur.data}]` + (cur.next ? ' -> ' : '');
        cur = cur.next;
    }
    console.log(s);
}

// Pengujian
console.log('=== Test Deteksi Siklus ===');
const A = buatList([1, 2, 3, 4, 5]);
console.log('Ada siklus (false):', adaSiklus(A));

// Buat siklus buatan
let temp = A;

```

```

while (temp.next) temp = temp.next;
temp.next = A.next;
console.log('Ada siklus (true):', adaSiklus(A));

console.log('\n=== Test Merge Terurut ===');
const L1 = buatList([1, 3, 5, 7, 9]);
const L2 = buatList([2, 4, 6, 8, 10]);
const merged = mergeTerurut(L1, L2);
cetakList(merged);

```

Output Terminal (02-11-lanjutan.js)

```

=== Test Deteksi Siklus ===
Ada siklus (false): false
Ada siklus (true): true

=== Test Merge Terurut ===
[1] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7] -> [8] -> [9] -> [10]

```

4. Latihan 2: Implementasi Stack Berbasis Linked List

Berdasarkan ketentuan Latihan 2, diimplementasikan struktur data Stack menggunakan Linked List secara komposisi dengan performa operasi $O(1)$ memanfaatkan teknik prepend/hapus head. Program disimulasikan menggunakan skenario Undo/Redo pengetikan.

Kode Program (03-stack-11.js)

```

// 03-stack-11.js
class Node {
  constructor(data) {
    this.data = data;
    this.next = null;
  }
}

class StackWithLinkedList {
  constructor() {
    this.head = null;
    this._size = 0;
  }

  push(data) {
    const newNode = new Node(data);
    newNode.next = this.head;
  }
}

```

```

        this.head = newNode;
        this._size++;
    }

    pop() {
        if (this.isEmpty()) return null;
        const poppedData = this.head.data;
        this.head = this.head.next;
        this._size--;
        return poppedData;
    }

    peek() {
        if (this.isEmpty()) return null;
        return this.head.data;
    }

    isEmpty() {
        return this.head === null;
    }

    get size() {
        return this._size;
    }

    print() {
        if (this.isEmpty()) {
            console.log('[Stack Kosong]');
            return;
        }
        let current = this.head;
        let result = '';
        while (current) {
            result += `[\${current.data}]` + (current.next ? ' -> ' :
'');
            current = current.next;
        }
        console.log(`TOP Stack: \${result}`);
    }
}

// Simulasi Undo / Redo

```

```

const undoStack = new StackWithLinkedList();
const redoStack = new StackWithLinkedList();

console.log('--- Pengguna Mengetik ---');
undoStack.push('Ketik "Halo"');
undoStack.push('Ketik "Dunia"');
undoStack.push('Ketik "!"');
undoStack.print();

console.log('\n--- Aksi UNDO ---');
let undoAction = undoStack.pop();
redoStack.push(undoAction);
undoStack.print();

console.log('\n--- Aksi REDO ---');
let redoAction = redoStack.pop();
undoStack.push(redoAction);
undoStack.print();

```

Output Terminal (03-stack-11.js)

```

--- Pengguna Mengetik ---
TOP Stack: [!] -> [Ketik "Dunia"] -> [Ketik "Halo"]

--- Aksi UNDO ---
TOP Stack: [Ketik "Dunia"] -> [Ketik "Halo"]

--- Aksi REDO ---
TOP Stack: [!] -> [Ketik "Dunia"] -> [Ketik "Halo"]

```

5. Commit dan Push ke GitHub

Langkah terakhir adalah melakukan penyimpanan seluruh berkas pengerjaan praktikum ke dalam penyimpanan cloud repository GitHub melalui rentetan instruksi berikut:

```

cd ..
git status
git add praktikum-6/
git commit -m "Praktikum 6: Linked List Lengkap dan Latihan"
git push origin main

```